

ALMOND

DeepWuKong Robustness Experiment System

INSTALLATION MANUAL

Verified Docker delivery, GPU setup, build, test, launch, and troubleshooting instructions.

SUBMISSION	2026 Term 2
PLATFORM	Windows + Docker Desktop + NVIDIA GPU
DOCUMENT VERSION	1.2 7 August 2026

Use this manual with the source-code submission and the verified UNSW OneDrive Docker delivery folder.

1 Installation overview

ALMOND measures the robustness of the DeepWuKong CWE-119 vulnerability detector under controlled source-code and program-graph perturbations.

Installation map

Step 1	Install and start Docker Desktop using Linux containers and the WSL 2 backend.
Step 2	Confirm that Docker can access a supported NVIDIA GPU.
Step 3	Save the DeepWuKong runtime TAR under <code>baselines\deepwukong\module_tranning</code> , verify it, and load it.
Step 4	Extract the ALMOND source-code ZIP to a writable Windows directory.
Step 5	Build the project image and run all 66 container tests.
Step 6	Start ALMOND with <code>Start.exe</code> or <code>robustness_experiments\Start.ps1</code> .

Included in the source submission

- Project source, Windows launcher, Docker definition, tests, input samples, checkpoint, dashboards, and representative evidence.
- The Docker runtime archive is not stored in Git or inside the Moodle source ZIP.
- The local `module_tranning` directory is ignored by Git and excluded from the Docker build context.
- Generated outputs remain on the host through the Docker Compose outputs bind mount.

Full model inference cannot run until the verified runtime TAR has been loaded under the expected Docker tag.

Repository-root entry points

```
.\Start.exe
.\robustness_experiments\Start.ps1
```

2 System requirements

Operating system	Windows 10 or Windows 11, 64-bit.
Container runtime	Docker Desktop with Linux containers and the WSL 2 backend.
GPU	NVIDIA GPU exposed to Docker. GPU access is required for full inference.
Driver	NVIDIA driver compatible with the CUDA 12.8 runtime in the supplied image.
Shell	Windows PowerShell 5.1 or PowerShell 7.
Network	Required to download the OneDrive archive and while the project build installs Graphviz.
Host Graphviz	Required only for host-side PDG Atlas rendering tests; the project Docker image installs it automatically.
Storage	Allow space for the 4.439 GiB TAR, extracted Docker layers, project image, and generated outputs.

Verify Docker Desktop

```
docker version
docker info
```

Verify host and container GPU access

```
nvidia-smi
docker run --rm --gpus all nvidia/cuda:12.8.0-base-ubuntu22.04 nvidia-smi
```

If the host GPU works but the container command fails, update Docker Desktop and the NVIDIA driver, then restart Windows.

3 Download and load the verified runtime

[Open the verified UNSW OneDrive Docker delivery folder](#)

From the extracted repository root, create the local-only directory below. In the browser download dialog, save the TAR there using the exact filename.

```
$RuntimeDir = ".\baselines\deepwukong\module_training"
New-Item -ItemType Directory -Force $RuntimeDir | Out-Null
```

Required local path: baselines\deepwukong\module_training\deepwukong-rtx5060-cu128-experimental.tar.
This directory is ignored by Git and is not copied into the project image.

Required archive

Filename	deepwukong-rtx5060-cu128-experimental.tar
Local path	baselines\deepwukong\module_training\deepwukong-rtx5060-cu128-experimental.tar
Size	4,766,494,208 bytes (4.439 GiB)
SHA-256	0482EA09F89569072427344B1DADA5E72878DF2E7BC99F878F5895B17DAF6B1D
Docker tag	deepwukong-rtx5060-cu128:experimental
Image ID	sha256:4735e489150a248ff4dc2040d366c5c09721263db9f6d8f7b116d39c0d035aea
Platform	linux/amd64

Verify the download

```
$RuntimeArchive = Join-Path $RuntimeDir "deepwukong-rtx5060-cu128-experimental.tar"
(Get-Item $RuntimeArchive).Length
(Get-FileHash $RuntimeArchive -Algorithm SHA256).Hash
```

The calculated SHA-256 must be exactly 0482EA09F89569072427344B1DADA5E72878DF2E7BC99F878F5895B17DAF6B1D. Do not load the archive if it differs.

Load and inspect the image

```
docker load -i $RuntimeArchive
docker image inspect deepwukong-rtx5060-cu128:experimental
```

A verified test import of this exact TAR restored the expected tag successfully.

4 Build and start ALMOND

Extract the source ZIP

Extract the complete source submission to a writable local directory. Do not run it directly from inside a ZIP. Open PowerShell in the repository root, which contains README.md and Start.exe.

Confirm required files

```
Test-Path .\Start.exe
Test-Path .\robustness_experiments\Start.ps1
Test-Path .\scripts\docker\compose.yaml
Test-Path .\baselines\deepwukong\models\deepwukong\deepwukong_cwe119_best.ckpt
Test-Path $RuntimeArchive
```

Build the project image

```
docker compose -f scripts/docker/compose.yaml build almond
```

The build creates t17a-almond:latest. The Dockerfile installs Graphviz automatically and excludes archived outputs and the large PDG Atlas page set from the build context.

Start the system

```
.\Start.exe

# Equivalent PowerShell entry point
.\robustness_experiments\Start.ps1
```

Start.exe builds when required, launches the interactive console, serves dashboards on port 8000, and preserves outputs on the host.

5 Test and inspect results

Run the complete container test suite

```
docker compose -f scripts/docker/compose.yaml run --rm almond tests
```

Verified result in the target Docker/GPU environment: Ran 66 tests - OK.

Run the archive and live-inference smoke test

```
.\Start.exe  
# Select 2. Run Smoke Test
```

The smoke test verifies the TAR's exact byte count and SHA-256 before running one baseline through Joern, PDG/XFG construction, and DeepWuKong inference.

Verify GPU access through ALMOND

```
docker run --rm --gpus all --entrypoint python `t17a-almond:latest` `c "import torch; print(torch.cuda.is_available())"
```

The output must be True. Use nvidia-smi on the host or in a CUDA container to inspect the detected GPU name.

Browser locations

Dashboard index	http://localhost:8000/outputs/index.html
PDG Atlas	http://localhost:8000/robustness_experiments/showcase/deepwukong_pdg_showcase.html
Final run	outputs/run_20260731_124703_code_all_input_sources/dashboard.html
Graph comparison	outputs/run_20260731_124703_code_all_input_sources/graph_comparison/dashboard.html

Use docker compose run rather than docker compose up for the interactive console because it requires direct terminal input.

6 Troubleshooting and assessor checklist

Docker daemon unavailable	Start Docker Desktop, wait for the Linux engine to report Running, then repeat docker version.
Checksum mismatch	Delete the incomplete download, download the TAR again from the documented OneDrive folder, and recheck SHA-256.
Runtime TAR missing	Save the exact TAR under baselines\deepwukong\module_training, then rerun the smoke test.
Base image not found	Repeat docker load and confirm that docker image inspect shows the exact expected tag.
GPU unavailable	Confirm nvidia-smi works on the host and in a CUDA container. Update Docker Desktop and the NVIDIA driver if required.
PowerShell blocked	Run Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass. Do not change the machine-wide policy.
Graphviz tests fail	For container tests, rebuild the project image. For host tests, install Graphviz and reopen the terminal.
Dashboard has no data	Confirm outputs exists on the host and the final run contains dashboard.html and summary.json.

Assessor final checklist

- The OneDrive folder opens and the 4.439 GiB TAR downloads successfully.
- The archive checksum and Docker tag match this manual.
- Docker Desktop and the NVIDIA GPU are available.
- The project image builds from the submitted source.
- All 66 container tests pass.
- Start.exe launches the console and the Dashboard, PDG Atlas, and graph comparison open.

Retain the OneDrive link, archive checksum, test output, and final successful launch evidence with the submission.